GA GUARDIAN

Nunchi

Genesis Vaults

Security Assessment

October 8th, 2025

Summary

Audit Firm Guardian

Prepared By Jakub Zmyslowski, Cosine

Client Firm Nunchi

Final Report Date October 8, 2025

Audit Summary

Nunchi engaged Guardian to review the security of their Nunchi Genesis Vaults. From the 17th of September to the 22nd of September, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report. **Note:** Fixes to remediation findings have not been reviewed by Guardian in this report.

Confidence Ranking

Given the number of High and Critical issues detected as well as additional code changes made after the main review, and to address findings from the remediation review. Guardian recommends that an independent security review of the protocol at a finalized frozen commit is conducted before deployment.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

📊 PoC test suite: https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975

Table of Contents

Project Overview

Project Summary

Project Name	Nunchi
Language	Solidity
Codebase	https://github.com/Nunchi-trade
Commit(s)	Initial commit: f1c9c4f785c5471485185b6b81d20039d7287077 Final commit: 494f3208c024bb11c7d97e7698fe9560dee0d69c

Audit Summary

Delivery Date	October 8, 2025
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	4	0	0	1	0	3
Medium	7	0	0	2	0	5
Low	6	0	0	4	0	2
Info	10	0	0	3	0	7

Audit Scope & Methodology

Scope and details:

contract,source,total,comment
genesis-vaults/src/tokens/GenesisVaultSYToken.sol,154,269,75
genesis-vaults/src/storage/GlobalStorage.sol,66,129,46
genesis-vaults/src/storage/Vault.sol,212,390,127
genesis-vaults/src/strategies/BaseGenesisStrategy.sol,19,43,16
genesis-vaults/src/strategies/NoYieldStrategy.sol,47,68,3
genesis-vaults/src/modules/ConfigurationModule.sol,68,145,54
genesis-vaults/src/modules/StrategyModule.sol,34,59,15
genesis-vaults/src/modules/VaultModule.sol,61,103,20
genesis-vaults/src/libraries/Errors.sol,30,46,11
source count: {
  total: 1252,
  source: 691,
  comment: 367,
  single: 191,
  block: 176,
  mixed: 2,
  empty: 196,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 0.5311143270622286
}

Also in scope: AAVEStrategy.sol

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	🔴 Critical	🟠 High	🟡 Medium
Likelihood: *Medium*	🟠 High	🟡 Medium	🟢 Low
Likelihood: *Low*	🟡 Medium	🟢 Low	🟢 Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	Global Withdrawal Delay For SY Holders	Logical Error	High	Resolved
H-02	Share Price Inflation Attack	Rounding	High	Acknowledged
H-03	Cross Vault Drains In No-Strategy Vaults	Logical Error	High	Resolved
H-04	Cross Vault Drains If Strategy Is Shared	Logical Error	High	Resolved
M-01	emergencyRecover Function Is Unreachable	Access Control	Medium	Resolved
M-02	Missing Withdrawal Limit	Validation	Medium	Resolved
M-03	SY Exchange Rate Mispricing	Logical Error	Medium	Resolved
L-01	Strategy Yield Distribution Sandwich Attack	MEV	Low	Acknowledged
L-02	Missing Event For Critical State Change	Best Practices	Low	Resolved
L-03	Harvest Flow May Be Capital Inefficient	Rewards	Low	Acknowledged
L-04	Missing Check In validateStrategy	Validation	Low	Acknowledged
L-05	Wrong Events In SY Token	Best Practices	Low	Resolved
I-01	Unsafe Operations	Best Practices	Info	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
I-02	Unused Code	Superfluous Code	Info	Resolved
I-03	Variables Not Marked As Immutable	Gas Optimization	Info	Resolved
I-04	ConfigurationModule Initialization	Frontrunning	Info	Acknowledged
I-05	Overly Permissive Fee Bounds	Best Practices	Info	Acknowledged
I-06	Incompatibility With Non-Standard ERC20s	Best Practices	Info	Acknowledged

H-01 | Global Withdrawal Delay For SY Holders

Category	Severity	Location	Status
Logical Error	High	Vault.sol: 112	Resolved

Description PoC

Users deposit underlying tokens through the SY wrapper, `GenesisVaultSYToken`. The wrapper calls deposit function on Vault module, where the logic records a cooldown timestamp for the caller address.

When the SY wrapper deposits, `msg.sender` is the SY contract address, so the vault stores one `lastDepositTime` for the entire wrapper.

When anyone later redeems via the SY wrapper, the wrapper calls withdraw on `VaultModule` contract. That path enforces the withdrawal delay against the caller address.

Because the cooldown is tracked once for the SY address (not per end-user), any deposit via the SY wrapper resets `lastDepositTime` and pushes out the withdrawal window for every SY holder.

An attacker can grief all SY redemptions by repeatedly depositing the minimum allowed amount just before the delay expires, creating a redemption DoS for all users who hold SY for that vault.

Recommendation

Move the cooldown enforcement to the SY wrapper on a per-user basis. `Track lastDepositTime[user]` inside the SY contract and enforce the delay on redeem. Skip the further withdrawal delay validation for registered SY wrappers.

Resolution

Nunchi Team: The issue was resolved in PR#4.

H-02 | Share Price Inflation Attack

Category	Severity	Location	Status
Rounding	High	VaultModule.sol: 23-49	Acknowledged

Description PoC

The protocol performs a check that the given user did not receive 0 shares and they have a configurable parameter to set a `minDeposit` amount for vaults.

These security checks will decrease impact and likelihood but depending on the configs and the given situation a share price inflation attack may still be possible.

Recommendation

Be aware and consider to further mitigate against share price inflation attacks by implementing virtual shares or minting shares directly in the init flow of a new vault.

Resolution

Nunchi Team: Acknowledged.

H-03 | Cross Vault Drains In No-Strategy Vaults

Category	Severity	Location	Status
Logical Error	High	Vault.sol: 301	Resolved

Description PoC

Vaults without a strategy route deposits to the Genesis Factory and price shares using `getTotalAssets`, which, in no-strategy mode, returns the Factory's entire balance of the vault's `liquidityToken`.

```
function getTotalAssets(
Data storage vault
) internal view returns (uint256) {
if (address(vault.strategy) == address(0)) {
return IERC20(vault.liquidityToken).balanceOf(address(this));
}
return IGenesisStrategy(vault.strategy).totalAssets();
}
```

Because this balance is shared across all no-strategy vaults that use the same token, a deposit into Vault B increases the Factory's balance and immediately inflates `getTotalAssets` for Vault A.

Conversions then use contaminated AUM value, letting redemptions in one vault withdraw assets funded by another, and mispricing new mints.

Recommendation

Use per-vault custody. Track each vault assets and use it in `getTotalAssets` instead of the Factory balance.

Resolution

Nunchi Team: The issue was resolved in PR#5.

H-04 | Cross Vault Drains If Strategy Is Shared

Category	Severity	Location	Status
Logical Error	High	Global	Resolved

Description PoC

Users are able to drain the factory if multiple vaults share the same strategy. The reason for this is that they both use the same summed up total amount for the share price calculation but a different `totalShare` value.

For example:

• Vault A is connected to Strategy A and 100 WETH is deposited

• Vault B is created and connected to Strategy A

• Both vaults share the same totalAssets amount (`balanceOf` the strategy contract)

• Eve deposits 1 WETH into Vault B and therefore owns 100% of the shares of Vault B

• Eve withdraws all of her shares and by doing so receives: `assetsOut = vaultBSharesIn * summedUpAssets / totalVaultBShares = 1e18 * 101e18 / 1e18 = 101e18`

Recommendation

Consider to enforce a 1 to 1 relationship between vaults and strategies.

Resolution

Nunchi Team: The issue was resolved in PR#5.

M-01 | emergencyRecover Function Is Unreachable

Category	Severity	Location	Status
Access Control	Medium	GenesisVaultSYToken.sol: 287-300	Resolved

Description

The `emergencyRecover` function in the `GenesisVaultSYToken` contract is unreachable as it has a `onlyFactory` modifier. But there is no contract in the factory to call it.

Recommendation

Consider adding a function to the factor to be able to call it.

Resolution

Nunchi Team: The issue was resolved in PR#8.

M-02 | Missing Withdrawal Limit

Category	Severity	Location	Status
Validation	Medium	Vault.sol: 100-125	Resolved

Description

The `AUDIT_SCOPE.md` document talks about a `Configurable deposit/withdrawal limits` in the `Key Features` of the vault. However only a configurable deposit limit was implemented.

Recommendation

Consider to implement a configurable withdrawal limit too or adjust the docs accordingly.

Resolution

Nunchi Team: The issue was resolved in PR#6.

M-03 | SY Exchange Rate Mispricing

Category	Severity	Location	Status
Logical Error	Medium	GenesisVaultSYToken.sol: 91-99	Resolved

Description PoC

Users can deposit directly into the Genesis vault (VaultModule.deposit) or via the SY wrapper (GenesisVaultSYToken.deposit). When depositing via SY, the Genesis factory mints vault shares to the SY contract, and the SY contract mints an equal amount of ERC-20 tokens to the user.

The SY token exposes an exchangeRate function that indicates the number of underlying assets per SY token. Because the vault allows direct deposits (EOAs mint vault shares that are not represented by SY supply), totalAssets includes all assets, while SY.totalSupply includes only SY-backed shares.

As soon as non-SY shares are outstanding, the exchangeRate becomes inflated. Any off-chain valuation, UI or DEX logic that uses exchangeRate as a price proxy will be incorrect, which may open further attack scenarios.

Recommendation

Compute the SY exchange rate as the price per share, using the vault's convertToAssets, so it reflects all outstanding shares (SY + direct holders).

Resolution

Nunchi Team: The issue was resolved in PR#4.

L-01 | Strategy Yield Distribution Sandwich Attack

Category	Severity	Location	Status
MEV	Low	VaultModule.sol: 23-76	Acknowledged

Description

A withdraw delay can be configured which reduces the likelihood but it may still be possible and profitable to front a yield distribution in the underlying vault of a strategy and deposit a lot of funds into the vault to gain most of the yield and exit right after the withdraw delay passed.

So, their may still be enough incentive for MEV here.

Recommendation

Be aware and consider one of these options:

• Always configure the withdraw delay accordingly

• Only pick vaults which do not have stepwise jumps in their share price value

• Implement a mechanism that distributes yield over time

Resolution

Nunchi Team: Acknowledged.

L-02 | Missing Event For Critical State Change

Category	Severity	Location	Status
Best Practices	Low	Global	Resolved

Description

Multiple functions which perform critical state changes do no emit an event:

• `ConfigurationModule:transferOwnership`

• `StrategyModule:setPerformanceFee`

Recommendation

Consider emitting events in all functions which perform critical state changes to follow best practices.

Resolution

Nunchi Team: The issue was resolved in commit 2f13bd2.

L-03 | Harvest Flow May Be Capital Inefficient

Category	Severity	Location	Status
Rewards	Low	Vault.sol: 494-515	Acknowledged

Description

The harvest flow is the following:

• All of the accrued yield is withdrawn

• The performance fee is collected

• The rest of the yield is deposited back into the vault

This is very capital inefficient if there is a withdraw and or deposit fee in the given vault.

Recommendation

Be aware and consider to not use vaults with deposit or withdraw fees or rewrite the logic in that case to only withdraw the performance fee.

Resolution

Nunchi Team: Acknowledged.

L-04 | Missing Check In validateStrategy

Category	Severity	Location	Status
Validation	Low	Vault.sol: 131-142	Acknowledged

Description

The `validateStrategy` function does not check if the given strategy contract uses `address(this)` as factory contract.

Using a strategy with a different address set as factory could lead to DoS or other major issues.

Recommendation

Consider implementing this check.

Resolution

Nunchi Team: Acknowledged.

L-05 | Wrong Events In SY Token

Category	Severity	Location	Status
Best Practices	Low	GenesisVaultSYToken.sol	Resolved

Description

The `GenesisVaultSYToken` emits events which do not strictly follow the `ERC-5115` SY token standard: https://eips.ethereum.org/EIPS/eip-5115

Recommendation

Consider to follow best practices and implement the same events as specified in the `ERC-5115` SY Token standard.

Resolution

Nunchi Team: The issue was resolved in PR#4.

I-01 | Unsafe Operations

Category	Severity	Location	Status
Best Practices	Info	Vault.sol: 485	Resolved

Description

There is a unsafe transfer call in the `collectPerformanceFee` function as well as unsafe approve operations globally.

Recommendation

Consider always using `safeTransfer` and `safeApprove` functions to follow best practices.

Resolution

Nunchi Team: The issue was resolved in PR#7.

I-02 | Unused Code

Category	Severity	Location	Status
Superfluous Code	Info	Global	Resolved

Description

There is unused code in multiple parts of the system.
Unused Errors:

• `NoFeeRecipient`

• `NoFeeConfigured`

• `OnlyRouter`

• `InvalidAsset`

• `InvalidRouter`

• `InsufficientBalance`

Unused Imports:

• `IGenesisStrategy` in the `VaultModule` contract

• `Errors` in the `GenesisVaultSYToken` contract

Recommendation

Consider to remove unused code.

Resolution

Nunchi Team: The issue was resolved in PR#7.

I-03 | Variables Not Marked As Immutable

Category	Severity	Location	Status
Gas Optimization	Info	GenesisVaultSYToken.sol: 22-28	Resolved

Description

In the `GenesisVaultSYToken` contract, the `vaultId`, factory, and asset variables assigned in the constructor are never modified afterward. There are no setter functions or internal assignments that change these values post-deployment.

Keeping them as regular public variables causes unnecessary storage reads at runtime, which consume gas, whereas declaring them immutable embeds the values in bytecode at deployment, eliminating storage access costs.

Recommendation

Declare the mentioned fields as immutable.

Resolution

Nunchi Team: The issue was resolved in PR#7.

I-04 | ConfigurationModule Initialization

Category	Severity	Location	Status
Frontrunning	Info	ConfigurationModule.sol: 20	Acknowledged

Description

The initialize function, implemented in `ConfigurationModule` is external and lacks access control. The first caller permanently sets `GlobalStorage.owner`, after which further calls revert.

If deployment and initialization are not atomic, an attacker can front-run the initialization and set themselves as owner, forcing redeployment.

Recommendation

Initialize the `ConfigurationModule` in the Genesis Factory's constructor during deployment to prevent front-running.

Resolution

Nunchi Team: Acknowledged.

I-05 | Overly Permissive Fee Bounds

Category	Severity	Location	Status
Best Practices	Info	Vault.sol: 157	Acknowledged

Description

The `validatePerformanceFee` function only rejects rate greater than 10000 BPS, so a 100% performance fee is currently permitted.

This is an overly permissive configuration that poses unexpected risk if set accidentally.

Recommendation

Enforce a sensible hard cap below 100% (e.g. 2000–3000 bps), and consider applying a time lock for fee increases.

Resolution

Nunchi Team: Acknowledged.

I-06 | Incompatibility With Non-Standard ERC20s

Category	Severity	Location	Status
Best Practices	Info	Global	Acknowledged

Description

The vault assumes standard `ERC-20` behaviour - exact amount is received on transfer, balances change only by that amount and math can safely rely on the requested value.

If the underlying token deviates from these assumptions, internal accounting can drift and downstream calls may operate on incorrect balances.

Examples include fee-on-transfer or deflationary tokens that deliver fewer tokens than requested, rebasing tokens whose balances change asynchronously or `ERC-777` hooks that may introduce the reentrancy risk.

Because the implementation mints shares from the requested amount instead of the observed balance delta, it can over-mint shares and attempt to deposit more into the strategy than it actually holds, breaking internal accounting.

Recommendation

Accept only standard `ERC-20` tokens or update the vault logic to correctly handle non-standard behaviours. Note that even commonly used tokens such as USDT or USDC could enable transfer fees in the future.

Resolution

Nunchi Team: Acknowledged.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Strategy Switch Resets Performance Fee	Rewards	Medium	Acknowledged
M-02	DoS Griefing Attack	DoS	Medium	Acknowledged
M-03	Withdraw Delay Can Be Avoided	Validation	Medium	Resolved
M-04	Function Is Unreachable	Access Control	Medium	Resolved
L-01	Mismatch In Migration Waiting Period	Documentation	Low	Acknowledged
I-01	Misleading Event In claimStrategyIncentives	Events	Info	Resolved
I-02	Unused Code	Superfluous Code	Info	Resolved
I-03	Unnecessary Try/Catch Blocks	Superfluous Code	Info	Resolved
I-04	Redundant Token Transfer	Best Practices	Info	Resolved

M-01 | Strategy Switch Resets Performance Fee

Category	Severity	Location	Status
Rewards	Medium	Vault.sol: 369	Acknowledged

Description

The `switchStrategy` flow calls the `unsetStrategy` function to unset the old strategy. However, this function also resets the `performanceFeeRate` and the `feeRecipient`. This may lead to lost revenue for the protocol.

Recommendation

Consider to not reset the performance fee configs or add a boolean if this is wished or not.

Resolution

Nunchi Team: Acknowledged.

M-02 | DoS Griefing Attack

Category	Severity	Location	Status
DoS	Medium	GenesisVaultSYToken.sol: 210	Acknowledged

Description

There is a withdrawal delay based on when a user was the recipient of a deposit the last time. However, an arbitrary address can be set as recipient of a deposit.

This allows users to DoS other users from withdrawing their funds by depositing a small amount of funds and set the victim as recipient.

Recommendation

As long as the `minDeposit` value is high enough the chance that this happens is unlikely. But be aware and consider to make changes if necessary.

Resolution

Nunchi Team: Acknowledged.

M-03 | Withdraw Delay Can Be Avoided

Category	Severity	Location	Status
Validation	Medium	GenesisVaultSYToken.sol	Resolved

Description

The withdraw delay can be avoided by transferring the tokens to another EOA and withdrawing with this account instead as the check is performed for the last deposit of the given `msg.sender`.

Recommendation

Here are two possible solutions:

• Make withdrawals a 2 step process with a delay in between

• Increase the `lastDepositTime` of the recipient during a transfer and add a `minTransfer` amount to mitigate against another griefing DoS vector

Resolution

Nunchi Team: The issue was resolved in PR#16.

M-04 | Function Is Unreachable

Category	Severity	Location	Status
Access Control	Medium	AAVEStrategy.sol: 302-327	Resolved

Description

The `emergencyWithdraw` function in the `AAVEStrategy` contract is unreachable as it has a onlyFactory modifier. But there is no contract in the factory to call it.

Recommendation

Consider adding a function to the factor to be able to call it.

Resolution

Nunchi Team: The issue was resolved in PR#16.

L-01 | Mismatch In Migration Waiting Period

Category	Severity	Location	Status
Documentation	Low	MigrationModule.sol: 22	Acknowledged

Description

The migration specification requires a 3-day waiting period before execution, but the implementation sets `MIN_MIGRATION_DELAY = 1 days` and applies `migrationDelay = 0 MIN_MIGRATION_DELAY : migrationDelay`.

Additionally, `GlobalStorage` comments state the default is 3 days. This inconsistency can allow migrations to execute earlier than stakeholders expect, reducing the intended review/exit window.

Recommendation

Align code and documentation by setting `MIN_MIGRATION_DELAY` to 3 days.

Resolution

Nunchi Team: Acknowledged.

I-01 | Misleading Event In claimStrategyIncentives

Category	Severity	Location	Status
Events	Info	StrategyModule.sol: 136	Resolved

Description

The `claimStrategyIncentives` function will pass and emit an `StrategyIncentivesClaimed` event even if no tokens were claimed.

Recommendation

Consider to revert in that case.

Resolution

Nunchi Team: The issue was resolved in PR#16.

I-02 | Unused Code

Category	Severity	Location	Status
Superfluous Code	Info	Global	Resolved

Description

There is unused code in multiple parts of the system:

• `IERC165` import in the `ConfigurationModule`

• `validateVaultInteraction` function in the `GlobalStorage` library

• `InsufficientBalance` error in the `AAVEStrategy` contract

• `safeTransfer` in the catch flow of the `AAVEStrategy` `deposit` function is redundant as the state will reset anyway on a revert

Recommendation

Consider to remove redundant code.

Resolution

Nunchi Team: The issue was resolved in PR#16.

I-03 | Unnecessary Try/Catch Blocks

Category	Severity	Location	Status
Superfluous Code	Info	AAVEStrategy.sol	Resolved

Description

`AAVEStrategy` contract uses try/catch around Aave calls in deposit, withdraw, and `withdrawAll` functions, where failures should revert the whole transaction.

This masks root causes by replacing Aave's revert data with generic errors and adds bytecode size and complexity with no functional benefit.

Recommendation

Consider removing try/catch pattern from deposit, withdraw and `withdrawAll` functions.

Resolution

Nunchi Team: The issue was resolved in PR#16.

I-04 | Redundant Token Transfer

Category	Severity	Location	Status
Best Practices	Info	AAVEStrategy.sol: 109	Resolved

Description

In `AAVEStrategy.deposit`, after pulling tokens from the factory, the catch block calls `safeTransfer` and then reverts.

Since the revert rolls back all state changes in this transaction, the transfer is redundant and adds an unnecessary external call.

Recommendation

Remove the `safeTransfer` in the catch block and revert directly.

Resolution

Nunchi Team: The issue was resolved in PR#16.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits